

الگوریتم دایجسترا (Dijkstra)

راهنمای آموزشی برای تدریس با گراف — همراه با یک مثال کامل و کمی سخت‌تر

قانون طلایی دایجسترا: همیشه برو سراغ نزدیک‌ترین گرهی که هنوز «قطعی» نکرده‌ای.

۱. ایده‌ی کلی در چند خط

هدف دایجسترا پیدا کردن کوتاه‌ترین فاصله از یک گرهی شروع تا همه‌ی گره‌های دیگر گراف است، وقتی وزن یال‌ها هیچ‌کدام منفی نیستند.

۱. یک دفترچه (جدول فاصله) بساز؛ فاصله‌ی گرهی شروع را صفر و بقیه را بی‌نهایت بگذار.
۲. از بین گره‌های «قطعی‌نشده»، نزدیک‌ترین را انتخاب کن.
۳. همسایه‌های آن گره را بررسی کن؛ اگر مسیر جدید کوتاه‌تر بود، فاصله را در دفترچه به‌روزرسانی کن (به این کار می‌گویند **relaxation** یا واهلش).
۴. آن گره را قطعی علامت بزن و دیگر برنگرد سراغش.
۵. تا وقتی گرهی قطعی‌نشده باقی مانده، مرحله‌ی ۲ تا ۴ را تکرار کن.

شبه‌کد ساده

```
function Dijkstra(graph, start):
    dist[start] = 0
    dist[v] = infinity   for every other node v
    visited = {}

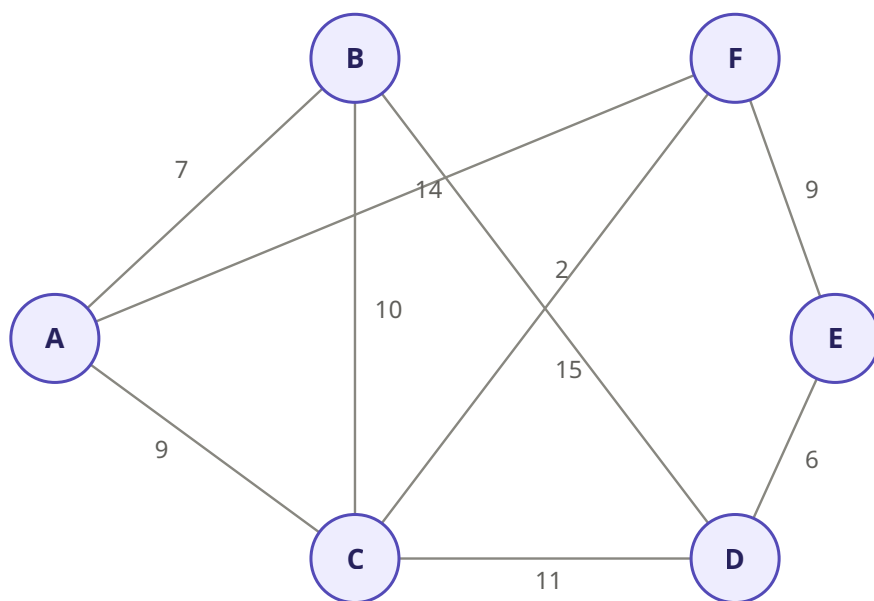
    while there are unvisited nodes:
        u = unvisited node with smallest dist[u]
        mark u as visited

        for each neighbor v of u:
            newDist = dist[u] + weight(u, v)
            if newDist < dist[v]:
                dist[v] = newDist

    return dist
```

۲. مثال سخت‌تر: گراف شش‌گره‌ای

این بار گراف بزرگ‌تر و شلوغ‌تر است تا شاگردت مجبور شود چند بار پشت سر هم دفترچه را به‌روزرسانی کند.



یال‌ها: $A-B=7$, $A-C=9$, $A-F=14$, $B-C=10$, $B-D=15$, $C-D=11$, $C-F=2$, $D-E=6$, $E-F=9$

۳. اجرای گام به گام از گره A

مرحله	گره انتخاب شده	A	B	C	D	E	F	توضیح کوتاه
۰	—	0	∞	∞	∞	∞	∞	دفترچه‌ی اولیه
۱	A	0	7	9	∞	∞	14	از A به B, C, F
۲	B (7)	0	7	9	22	∞	14	مسیر $B \rightarrow C$ بدتر بود ($9 < 17$)، پس رد شد؛ فقط D ثبت شد
۳	C (9)	0	7	9	20	∞	11	هم D و هم F کوتاه‌تر شدند
۴	F (11)	0	7	9	20	20	11	از F به E رسیدیم: $20 = 9 + 11$
۵	D (20)	0	7	9	20	20	11	مسیر $D \rightarrow E$ بهتر نبود ($20 < 26$)
۶	E (20)	0	7	9	20	20	11	همه قطعی شدند — تمام!

نکته‌ی مهم برای تدریس: در مرحله‌ی ۲، مسیر $B \rightarrow C$ را بررسی کردیم ($17 = 10 + 7$) ولی چون از فاصله‌ی فعلی C یعنی ۹ بیشتر بود، هیچ تغییری ندادیم. این دقیقاً همان بخشی از الگوریتم است که شاگردها معمولاً فراموش می‌کنند: relaxation یعنی «فقط وقتی به‌روزرسانی کن که مسیر جدید واقعاً کوتاه‌تر باشد»، نه هر بار.

۴. نتیجه‌ی نهایی و بازسازی مسیرها

مسیر	کوتاه‌ترین فاصله از A	گره
$A \rightarrow B$	7	B
$A \rightarrow C$	9	C
$A \rightarrow C \rightarrow F$	11	F
$A \rightarrow C \rightarrow D$	20	D
$A \rightarrow C \rightarrow F \rightarrow E$	20	E

نکته‌ی جالب: $A \rightarrow F$ مستقیم وزنش ۱۴ بود، ولی از مسیر $A \rightarrow C \rightarrow F$ فقط ۱۱ شد (۲+۹). این خیلی خوب نشان می‌دهد که کوتاه‌ترین مسیر همیشه مستقیم‌ترین مسیر نیست.

۵. چند نکته‌ی آموزشی برای کلاس

مفاهیمی که حتماً باید جا بیفتد

- چرا وقتی گره‌ای «قطعی» شد، دیگر برنمی‌گردیم سراغش.
- چرا این تضمین فقط وقتی برقرار است که همه‌ی وزن‌ها مثبت باشند.
- تفاوت relaxation (فقط وقتی بهتر بود آپدیت کن) با نوشتن هر عددی که پیدا کردیم.

تمرین پیشنهادی

- همین گراف را از گره F شروع کن و بخواه شاگرد جدول را خودش پر کند.
- یک یال با وزن منفی اضافه کن و بپرس چرا الگوریتم دیگر درست کار نمی‌کند.
- از او بخواه با یک صف اولویت (min-heap) پیچیدگی زمانی را حساب کند.

تهیه‌شده برای آموزش شبکه و مسیریابی — الگوریتم دایجسترا پایه‌ی پروتکل‌های Link-State مثل OSPF است.